

Paralelne i distribuirane arhitekture i jezici -
Seminarski rad

Davor Homa E2 53/2025

14. januar 2026.

Sadržaj

1	Uvod	3
2	Teorijska osnova	4
2.1	Matrica	4
2.2	Operacije nad matricama	4
2.2.1	Sabiranje	4
2.2.2	Oduzimanje	4
2.2.3	Množenje	4
2.3	Standardni iterativni algoritam za množenje matrica	4
2.4	Zavadi-pa-vladaj algoritam za množenje matrica	5
2.5	Štrasenov algoritam za množenje matrica	6
3	Implementacija algoritama	7
3.1	Implementacija u Rust-u	7
3.1.1	Iterativni algoritam za množenje matrica	7
3.1.2	Zavadi-pa-vladaj algoritam za množenje matrica	8
3.1.3	Štrasenov algoritam za množenje matrica	9
3.2	Implementacija u Go-u	11
3.2.1	Iterativni algoritam za množenje matrica	11
3.2.2	Zavadi-pa-vladaj algoritam za množenje matrica	11
3.2.3	Štrasenov algoritam za množenje matrica	13
4	Poređenje implementacija	15
4.1	Brzina i lakoća razvoja	15
4.2	Vreme izvršavanja	15
4.2.1	Iterativni algoritam	15
4.2.2	Zavadi-pa-vladaj algoritam	16
4.2.3	Štrasenov algoritam	17
4.2.4	Diskusija rezultata	18
4.3	Utrošak resursa	18
4.3.1	Utrošak zauzeća procesora	19
4.3.2	Utrošak memorije	19
5	Zaključak	20

1 Uvod

Množenje matrica predstavlja jednu od osnovnih operacija u numeričkim i naučnim proračunima i ima široku primenu u oblastima kao što su računarska grafika, obrada signala, mašinsko učenje, simulacije i rešavanje sistema linearnih jednačina. Zbog svoje vremenske složenosti i česte upotrebe nad velikim skupovima podataka, ova operacija je čest predmet optimizacije i paralelizacije, kao i pogodan primer za poređenje različitih algoritamskih pristupa i programskih jezika.

Standardni iterativni algoritam za množenje matrica ima vremensku složenost reda $O(n^3)$, što ga čini neefikasnim za velike dimenzije matrica. Zbog toga su razvijeni napredniji algoritmi, poput zavadi-pa-vladaaj pristupa i Štrasenovog algoritma, koji omogućavaju bolje iskorišćenje memorijskih resursa i smanjenje broja osnovnih operacija. Pored toga, savremeni hardver nudi mogućnost paralelnog izvršavanja, čime se dodatno mogu unaprediti performanse ovih algoritama.

Cilj ovog seminarskog rada jeste poređenje programskih jezika Rust i Go na primeru implementacije algoritama za množenje matrica.

U radu su implementirana tri algoritma za množenje matrica: standardni iterativni algoritam, zavadi-pa-vladaaj algoritam i Štrasenov algoritam. Drugi i treći algoritam implementirani su u paralelnoj izvedbi, kako bi se ispitala mogućnost iskorišćenja višeprocorskih sistema. Implementacije su realizovane u oba programska jezika, uz korišćenje njihovih standardnih i dodatnih biblioteka za paralelno izvršavanje.

Pored same implementacije, u radu se vrši poređenje jezika Rust i Go sa aspekta brzine i lakoće razvoja, kao i sa aspekta performansi, uključujući vreme izvršavanja i utrošak sistemskih resursa. Eksperimentalni rezultati dobijeni merenjem vremena izvršavanja za različite dimenzije matrica služe kao osnova za analizu prednosti i nedostataka oba jezika.

Struktura rada je organizovana tako da se najpre predstavi teorijska osnova algoritama za množenje matrica, a potom detaljan opis implementacija u oba jezika. Zatim sledi eksperimentalno poređenje i diskusija rezultata, dok su u zaključku sumirani glavni nalazi rada.

2 Teorijska osnova

2.1 Matrica

Matrica je pravougaona tabela brojeva ili elemenata iz nekog skupa organizovana u redove i kolone. Predstavlja osnovnu strukturu podataka u linearnim sistemima i numeričkim proračunima.

Za pravougaonu šemu brojeva a_{ij} , gde su $i = 1, 2, \dots, m$ a $j = 1, 2, \dots, n$, matrica dimenzija $m \times n$ može se zapisati kao [1]:

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & & & a_{2n} \\ \vdots & & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

2.2 Operacije nad matricama

2.2.1 Sabiranje

Sabiranje matrica je binarna operacija između dve matrice isključivo istih tipova ($m \times n$), čiji rezultat proizvodi novu matricu istog tipa, pri čemu se svaki element računa kao zbir odgovarajućih elemenata [1]:

$$c_{ij} = a_{ij} + b_{ij}, \quad i = 1, \dots, m, \quad j = 1, \dots, n.$$

2.2.2 Oduzimanje

Oduzimanje matrica se vrši analogno sabiranju, pri čemu se svaki element nove matrice računa po formuli [1]:

$$c_{ij} = a_{ij} - b_{ij}, \quad i = 1, \dots, m, \quad j = 1, \dots, n.$$

2.2.3 Množenje

Množenje matrica je binarna operacija između dve matrice koja rezultira novom matricom čiji elementi zavise od skalarnih proizvoda redova prve i kolona druge matrice. Za dve matrice A i B dimenzija $A \in \mathbb{R}^{m \times n}$ i $B \in \mathbb{R}^{n \times p}$, proizvod $C = A \cdot B$ je definisan samo ako broj kolona prve matrice odgovara broju redova druge matrice. Rezultujuća matrica C biće dimenzija $m \times p$. Element c_{ij} te matrice se računa kao [1]:

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj}, \quad i = 1, \dots, m, \quad j = 1, \dots, p$$

gde su a_{ik} elementi matrice A , a b_{kj} elementi matrice B .

2.3 Standardni iterativni algoritam za množenje matrica

Standardni iterativni algoritam za množenje matrica direktno proizlazi iz matematičke definicije matrice $C = A \cdot B$. Ideja je da se svaki element rezultujuće matrice C izračuna kao skalarni proizvod odgovarajućeg reda matrice A i kolone matrice B .

Ovaj algoritam se može implementirati u kodu koristeći tri ugnježdene petlje. Spoljašnje dve petlje prolaze kroz sve redove i kolone rezultujuće matrice C , dok unutrašnja petlja izvršava skalarni proizvod tako što množi odgovarajuće elemente iz reda matrice A i kolone matrice B i sabira rezultate.

Pseudo-kod algoritma je dat u nastavku.

```

for  $i = 1$  to  $n$  do
  for  $j = 1$  to  $p$  do
     $\text{sum} \leftarrow 0$ 
    for  $k = 1$  to  $m$  do
       $\text{sum} \leftarrow \text{sum} + A_{ik} \cdot B_{kj}$ 
    end for
     $C_{ij} \leftarrow \text{sum}$ 
  end for
end for

```

Glavna karakteristika ovog algoritma jeste njegova vremenska složenost. Kod kvadratnih matrica dimenzija $n \times n$, tri ugnježdene petlje vrše ukupno n^3 iteracija, gde svaka iteracija sadrži jednu operaciju množenja i sabiranja. To znači da ovaj algoritam ima vremensku složenost $O(n^3)$ i zbog toga nije pogodan za množenje matrica većih dimenzija.

2.4 Zavadi-pa-vladaj algoritam za množenje matrica

Zavadi-pa-vladaj (engl. *divide and conquer*) algoritam rasterećuje problem tako što originalni zadatak podeli na manje, nezavisne podprobleme koji se rešavaju rekursivno i na kraju se njihovi rezultati kombinuju kako bi se dobio konačan rezultat [2].

U kontekstu množenja kvadratnih matrica dimenzija $n \times n$, zavadi-pa-vladaj algoritam počinje podelom ulaznih matrica A i B na četiri podmatrice veličine $n/2 \times n/2$. Matrice se pišu u blok formi:

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

Rezultujuća matrica $C = A + B$ se takođe deli na četiri podmatrice C_{11} , C_{12} , C_{21} , C_{22} . Matrični proizvod se tada može izraziti kroz manje proizvode i sabiranja:

$$\begin{aligned} C_{11} &= A_{11}B_{11} + A_{12}B_{21}, \\ C_{12} &= A_{11}B_{12} + A_{12}B_{22}, \\ C_{21} &= A_{21}B_{11} + A_{22}B_{21}, \\ C_{22} &= A_{21}B_{12} + A_{22}B_{22}. \end{aligned}$$

Ovaj algoritam množenja matrica koristi osam rekursivnih množenja podmatrica veličine $n/2 \times n/2$ i četiri sabiranja rezultata za formiranje završnih blokova rezultujuće matrice C .

Ako označimo vreme izvršenja ovog algoritma sa $T(n)$, rekurentna formula glasi:

$$T(n) = 8T(n/2) + \Theta(n^2)$$

gde je:

- $8T(n/2)$ posledica osam rekursivnih poziva,
- $\Theta(n^2)$ vreme potrebno za sabiranje rezultata podmatrica.

Primenom master teoreme, dobijamo da ova rekurzija daje složenost $O(n^3)$, što je jednako složenosti standardnog iterativnog algoritma. Sledi da, bez dodatnih optimizacija, ovaj osnovni zavadi-pa-vladaj pristup ne poboljšava vremensku složenost u odnosu na naivnu metodu.

2.5 Štrasenov algoritam za množenje matrica

Štrasenov algoritam prvi put je predstavio nemački matematičar Volker Strassen 1969. godine, dokazavši da tradicionalni algoritam sa složenošću $O(n^3)$ nije optimalan.

Osnovna ideja Štrasenovog algoritma zasniva se na pristupu zavadi-pa-vladaaj, ali uz ključnu optimizaciju koja smanjuje broj množenja podmatrica. Ako se kvadratne matrice A i B dimenzija $n \times n$ podele u četiri podmatrice dimenzija $n/2 \times n/2$, standardni zavadi-pa-vladaaj pristup zahteva osam rekurzivnih množenja podmatrica kako bi se izračunali blokovi rezultujuće matrice C . Štrasenov algoritam umesto toga koristi sedam rekurzivnih množenja uz dodatne operacije sabiranja i oduzimanja matrica [3].

Algoritam definiše sledećih sedam pomoćnih matrica:

$$\begin{aligned}M_1 &= (A_{11} + A_{22}) \cdot (B_{11} + B_{22}), \\M_2 &= (A_{21} + A_{22}) \cdot B_{11}, \\M_3 &= A_{11} \cdot (B_{12} - B_{22}), \\M_4 &= A_{22} \cdot (B_{21} - B_{11}), \\M_5 &= (A_{11} + A_{12}) \cdot B_{22}, \\M_6 &= (A_{21} - A_{11}) \cdot (B_{11} + B_{12}), \\M_7 &= (A_{12} - A_{22}) \cdot (B_{21} + B_{22})\end{aligned}$$

Značajno je naglasiti da se za izračunavanje svake od ovih matrica koristi tačno jedno množenje matrica. Takođe, ove matrice se mogu izračunati nezavisno jedna od druge, što omogućava paralelizaciju ovog algoritma.

Na osnovu ovih proizvoda, blokovi rezultujuće matrice C se kombinuju na sledeći način:

$$\begin{aligned}C_{11} &= M_1 + M_4 - M_5 + M_7, \\C_{12} &= M_3 + M_5, \\C_{21} &= M_2 + M_4, \\C_{22} &= M_1 + M_3 - M_2 + M_6\end{aligned}$$

Navedeni proces primenjuje se rekurzivno dok se ne stigne do najnižeg nivoa.

Zbog smanjenja broja rekurzivnih množenja sa osam na sedam, Štrasenov algoritam dostiže bolju asimptotsku vremensku složenost od predhodna dva pristupa:

$$T(n) = 7T(n/2) + O(n^2) \implies O(n^{\ln 7}) \approx O(n^{\ln 2.8074}),$$

što predstavlja značajno poboljšanje u odnosu na $O(n^3)$ za velike dimenzije matrica.

3 Implementacija algoritama

U ovom poglavlju opisuju se konkretne implementacije algoritama za množenje matrica u programskim jezicima Rust i Go.

3.1 Implementacija u Rust-u

Za implementaciju svih algoritama u Rust-u korišćene su dve osnovne strukture: *Matrix* i *MatrixView*.

Struktura *Matrix* sadrži broj redova, broj kolona i jednodimenzionalni vektor elemenata [Listing 1].

```
pub struct Matrix {  
    pub rows: usize,  
    pub cols: usize,  
    pub data: Vec<i32>,  
}
```

Listing 1: Struktura *Matrix*

Omogućene su osnovne operacije kreiranja matrice i pristupa elementima matrice [Listing 2].

```
impl Matrix {  
    pub fn new(rows: usize, cols: usize) -> Self { ... }  
    pub fn get(&self, r: usize, c: usize) -> i32 { ... }  
    pub fn set(&mut self, r: usize, c: usize, val: i32) { ... }  
}
```

Listing 2: Osnovne operacije nad Matricom

MatrixView predstavlja pogled na matricu ili njen poddeo, što je korisno za implementaciju zavadi-pa-vladaaj i Štrasenovog algoritma [Listing 3].

```
pub struct MatrixView<'a> {  
    pub data: &'a [i32],  
    pub rows: usize,  
    pub cols: usize,  
    pub stride: usize,  
}
```

Listing 3: Struktura *MatrixView*

Korišćenjem *MatrixView* strukture omogućava se efikasan rad sa podmatricama bez dodatnog kopiranja podataka, što je ključ za optimizaciju performansi pri paralelnoj obradi.

3.1.1 Iterativni algoritam za množenje matrica

Standardni iterativni algoritam za množenje matrica, opisan u poglavlju 2.3, implementiran je sa ciljem da se što direktnije prenesu matematička definicija u kod.

Funkcija *multiply_matrix* kreira novu mutabilnu matricu *C* koja služi za smeštanje rezultata proizvoda matrica *A* i *B* i kao povratna vrednost ove funkcije [Listing 4].

```

pub fn multiply_matrix(a: MatrixView, b: MatrixView) -> Matrix {
    assert!(a.cols == b.rows, "Incompatible_dimensions");
    let mut c = Matrix::new(a.rows, b.cols);
    for i in 0..a.rows {
        for k in 0..a.cols {
            for j in 0..b.cols {
                let v = c.get(i, j) + a.get(i, k) * b.get(k, j);
                c.set(i, j, v);
            }
        }
    }
    c
}

```

Listing 4: Rust - funkcija *multiply_matrix*

Proizvod matrica računa se pomoću tri ugnježdene petlje, gde se pomoću:

- i iterira kroz redove matrice A ,
- k iterira kroz kolone matrice B ,
- j iterira kroz kolone matrice C .

3.1.2 Zavadi-pa-vladaj algoritam za množenje matrica

Zavadi-pa-vladaj algoritam implementiran u Rust-u u ovom radu koristi rekurzivni pristup koji deli originalni problem množenja matrica na manje, nezavisne podprobleme. Kao što je objašnjeno u poglavlju 2.4, ulazne matrice dimenzija $n \times n$ se dele na četiri podmatrice, a konačan proizvod se formira sabiranjem rezultata manjih množenja.

Za kreiranje podmatrica korišćena je funkcija *split_view*, koja za datu matricu vraća četiri pogleda (*MatrixView*) na odgovarajuće kvadrante [Listing 5].

```

pub fn split_view<'a>(m: MatrixView<'a>) ->
    (MatrixView<'a>, MatrixView<'a>, MatrixView<'a>, MatrixView<'a>) {
    let r2 = m.rows / 2;
    let c2 = m.cols / 2;
    let a11 = MatrixView {
        data: m.data,
        rows: r2,
        cols: c2,
        stride: m.stride,
    };
    let a12 = ...
    let a21 = ...
    let a22 = MatrixView {
        data: &m.data[r2 * m.stride + c2..],
        rows: m.rows - r2,
        cols: m.cols - c2,
        stride: m.stride,
    };
    (a11, a12, a21, a22)
}

```

Listing 5: Rust - funkcija *split_view*

U implementaciji je uvedena konstanta `THRESHOLD` koja definiše prag pri kojem se u algoritmu prekida rekurzija i prelazi na standardno iterativno množenje

matrica. Ovaj prag je odabran zbog toga što je za manje matrice utrošak resursa znatno efikasniji kod iterativnog množenja u odnosu na rekurzivni pristup.

Glavna rekurzivna funkcija *divide_and_conquer* ima ulogu upravljanja algoritmom. Ona proverava bazni slučaj rekurzije, delegira paralelno izračunavanje blokova rezultujuće matrice i na kraju kombinuje dobijene podmatrice u jednu celinu [Listing 6].

```
pub fn divide_and_conquer(a: MatrixView, b: MatrixView) -> Matrix {
    let n = a.rows;

    if n <= THRESHOLD {
        return multiply_matrix(a, b);
    }

    let (c11, c12, c21, c22) = compute_blocks_parallel(a, b);
    merge_quadrants(&c11, &c12, &c21, &c22)
}
```

Listing 6: Rust - funkcija *divide_and_conquer*

Funkcija *compute_blocks_parallel* najpre ulazne matrice A i B deli na podmatrice pomoću funkcije *split_view*, a zatim implementira paralelno izračunavanje četiri bloka rezultujuće matrice $C_{11}, C_{12}, C_{21}, C_{22}$. Za paralelizaciju se koristi biblioteka *Rayon*, konkretno funkcija *rayon::join* [Listing 7].

```
fn compute_blocks_parallel(a: MatrixView, b: MatrixView) -> (Matrix, Matrix,
    Matrix, Matrix) {
    let (a11, a12, a21, a22) = split_view(a);
    let (b11, b12, b21, b22) = split_view(b);

    let ((c11, c12), (c21, c22)) = rayon::join(
        || rayon::join(
            || compute_block(a11, b11, a12, b21),
            || compute_block(a11, b12, a12, b22)),
        || rayon::join(
            || compute_block(a21, b11, a22, b21),
            || compute_block(a21, b12, a22, b22))
    );

    (c11, c12, c21, c22)
}
```

Listing 7: Rust - funkcija *compute_blocks_parallel*

Funkcija *compute_block* računa jedan blok rezultujuće matrice [Listing 8].

```
fn compute_block(a1: MatrixView, b1: MatrixView, a2: MatrixView, b2:
    MatrixView) -> Matrix {
    let (t1, t2) = rayon::join(
        || divide_and_conquer(a1, b1),
        || divide_and_conquer(a2, b2),
    );
    add_vec(&t1, &t2)
}
```

Listing 8: Funkcija *compute_block*

3.1.3 Štrasenov algoritam za množenje matrica

Štrasenov algoritam u Rust-u implementiran je kroz glavnu funkciju *strassen*, koja upravlja celokupnim algoritmom [Listing 9]. Za matrice dimenzija manje od

definisanog praga `THRESHOLD`, koristi se standardno iterativno množenje. Za veće matrice, poziva se funkcija `split_view` koja deli ulazne matrice na četiri podmatrice. Nakon toga se paralelno računanje sedam pomoćnih matrica delegira funkciji `compute_matrices_parallel`. Konačni rezultat se pomoću funkcije `combine_result` kombinuje u četiri kvadranta rezultujuće matrice.

```
pub fn strassen(a: MatrixView, b: MatrixView) -> Matrix {
    let n = a.rows;

    if n <= THRESHOLD {
        return multiply_matrix(a, b);
    }

    let a_blocks = split_view(a);
    let b_blocks = split_view(b);
    let matrices = compute_matrices_parallel(a_blocks, b_blocks);
    combine_result(matrices)
}
```

Listing 9: Rust - funkcija `strassen`

Funkcija `compute_matrices_parallel` računa sedam pomoćnih matrica koje Štrassenov algoritam koristi za smanjenje broja rekurzivnih množenja. Svaka matrica se računa paralelno korišćenjem *Rayon* biblioteke, konkretno funkcija `rayon::scope` i `rayon::spawn`.

```
fn compute_matrices_parallel(a: (MatrixView, MatrixView, MatrixView,
    MatrixView), b: (MatrixView, MatrixView, MatrixView, MatrixView))
    -> [Matrix; 7] {
    let (a11, a12, a21, a22) = a;
    let (b11, b12, b21, b22) = b;

    let mut m1 = Matrix::new(a11.rows, a11.cols);
    // kreiranje m2, m3, m4, m5, m6
    let mut m7 = Matrix::new(a11.rows, a11.cols);

    rayon::scope(|s| {
        s.spawn(|_| m1 = strassen(
            add_vec(&a11.to_matrix(), &a22.to_matrix()).view(),
            add_vec(&b11.to_matrix(), &b22.to_matrix()).view()
        ));
        // s.spawn(...) za m2 ... m6
        s.spawn(|_| m7 = strassen(
            sub_vec(&a12, &a22).view(),
            add_vec(&b21.to_matrix(), &b22.to_matrix()).view()
        ));
    });

    [m1, m2, m3, m4, m5, m6, m7]
}
```

Listing 10: Rust - funkcija `compute_matrices_parallel`

Funkcija `combine_result` uzima sedam matrica i kombinuje ih u četiri kvadranta rezultujuće matrice [Listing 11], dok funkcija `merge_quadrants` spaja kvadrante u jednu matricu dimenzija $n \times n$.

```

fn combine_result(m: [Matrix; 7]) -> Matrix {
    let c11 = add_vec(&sub_vec(&add_vec(&m[0], &m[3]).view(), &m[4].view()),
        &m[6]);
    let c12 = add_vec(&m[2], &m[4]);
    let c21 = add_vec(&m[1], &m[3]);
    let c22 = add_vec(&sub_vec(&add_vec(&m[0], &m[2]).view(), &m[1].view()),
        &m[5]);

    merge_quadrants(&c11, &c12, &c21, &c22)
}

```

Listing 11: Rust - funkcija *combine_result*

3.2 Implementacija u Go-u

Za razliku od implementacije u programskom jeziku Rust, u kojoj je bilo potrebno eksplicitno definisati strukture za rad sa matricama, implemetacija u programskom jeziku Go koristi ugrađene strukture podataka.

U Go-u su matrice predstavljene kao dvodimenzionalni nizovi, što omogućava direktan i jednostavan pristup elementima matrice bez potrebe za definisanjem dodatnih pomoćnih struktura. Elementima matrice pristupa se korišćenjem indeksne notacije $a[i][j]$.

U cilju povećanja čitljivosti koda i postizanja veće sličnosti sa implementacijom u Rust-u, u Go-u je uveden alias tipa za matricu: **Matrix** `[][]int`.

3.2.1 Iterativni algoritam za množenje matrica

Iterativni algoritam u za množenje matrica u Go-u implementiran je u skladu sa matematičkom definijom pomoću funkcije *multiplyMatrix*. Funkcija prima tri matrice kao parametre: ulazne matrice A i B , kao i matricu C u koju se upisuje rezultat množenja.

```

func multiplyMatrix(A, B, C Matrix) {
    ...

    for i := range rowsA {
        for k := range colsA {
            for j := range colsB {
                C[i][j] += A[i][k] * B[k][j]
            }
        }
    }
}

```

Listing 12: Go - funkcija *multiplyMatrix*

Proizvod matrica računa se pomoću tri ugnježdene petlje, gde se pomoću:

- i iterira kroz redove matrice A ,
- k iterira kroz kolone matrice B ,
- j iterira kroz kolone matrice C .

3.2.2 Zavadi-pa-vladaj algoritam za množenje matrica

Zavadi-pa-vladaj algoritam za množenje matrica u Go jeziku implementiran je krišćenjem rekurzivnog pristupa, analogno implementaciji u Rust-u. Osnovna ideja

algoritma ostaje ista: ulazne matrice dimenzija $n \times n$ dele se na četiri podmatrice, a rezultat se dobija kombinovanjem proizvoda odgovarajućih blokova, kao što je opisano u poglavlju 2.4.

Za razliku od Rust implementacije, u Go-u nije uvedena posebna struktura poput *MatrixView*. Umesto toga, podmatrice se predstavljaju kao pogledi nad postojećim dvodimenzionalnim nizovima, koristeći sečene (*slice*) nizove. Na ovaj način omogućeno je da se sa podmatricama radi bez kopiranja podataka, što je važno za efikasnost algoritma.

Podela matrice na četiri kvadranta realizovana je funkcijom *splitView*, koja za datu matricu vraća četiri podmatrice. Svaka od ovih podmatrica deli istu osnovnu memoriju sa originalnom matricom, čime se izbegava dodatna alokacija memorije.

```
func splitView(m Matrix) (a, b, c, d Matrix) {
    rows := len(m)
    cols := len(m[0])

    r2 := rows / 2
    c2 := cols / 2

    a = make(Matrix, r2)
    for i := 0; i < r2; i++ {
        a[i] = m[i]:c2
    }

    b = ...
    c = ...
    d = ...
}
```

Listing 13: Go - funkcija *splitView*

Glavna rekurzivna funkcija *divideAndConquer* upravlja izvršavanjem algoritma [Listing 14]. Funkcija najpre proverava bazni slučaj rekurzije. Ukoliko je dimenzija matrice manja ili jednaka unapred definisanom pragu THRESHOLD, primenjuje se iterativni algoritam za množenje matrica. Kao i kod implementacije u Rust-u, ovakav pristup je opravdan, jer je za male dimenzije iterativni algoritam efikasniji od rekurzivnog zbog manjeg troška upravljanja rekurzijom. U suprotnom slučaju, izračunavanje se delegira funkciji *computeBlocksParallel*.

```
func divideAndConquer(A, B, C Matrix) {
    n := len(A)
    if n <= THRESHOLD {
        multiplyMatrix(A, B, C)
    } else {
        computeBlocksParallel(A, B, C, n)
    }
}
```

Listing 14: Go - funkcija *divideAndConquer*

Funkcija *computeBlocksParallel* vrši paralelno računanje četiri bloka rezultujuće matrice C_{11} , C_{12} , C_{21} , C_{22} [Listing 15]. Paralelizacija je ostvarena korišćenjem gorutina (engl. *goroutines*) i sinhronizacionog mehanizma *sync.WaitGroup*, kojim se obezbeđuje da se sva četiri bloka izračunaju pre nastavka izvršavanja algoritma.

```

func computeBlocksParallel(A, B, C Matrix, n int) {
    a11, a12, a21, a22 := splitView(A)
    b11, b12, b21, b22 := splitView(B)
    c11, c12, c21, c22 := splitView(C)

    var wg sync.WaitGroup
    wg.Add(4)
    go func() { computeBlock(a11, b11, a12, b21, c11, n/2); wg.Done() }()
    go func() { computeBlock(a11, b12, a12, b22, c12, n/2); wg.Done() }()
    go func() { computeBlock(a21, b11, a22, b21, c21, n/2); wg.Done() }()
    go func() { computeBlock(a21, b12, a22, b22, c22, n/2); wg.Done() }()
    wg.Wait()
}

```

Listing 15: Go - funkcija *computeBlocksParallel*

Svaki blok rezultujuće matrice računa se funkcijom *computeBlock* [Listing 16].

```

func computeBlock(a1, b1, a2, b2, cBlock Matrix, size int) {
    t1 := makeZeroMatrix(size)
    t2 := makeZeroMatrix(size)

    var wg sync.WaitGroup
    wg.Add(2)
    go func() { divideAndConquer(a1, b1, t1); wg.Done() }()
    go func() { divideAndConquer(a2, b2, t2); wg.Done() }()
    wg.Wait()

    add(t1, t2, cBlock)
}

```

Listing 16: Go - funkcija *computeBlock*

3.2.3 Štrassenov algoritam za množenje matrica

Štrassenov algoritam za množenje matrica u jeziku Go implementiran je kroz glavnu funkciju *strassen* [Listing 17]. Kao i u prethodnim implementacijama, uvedena je konstanta **THRESHOLD** koja definiše prag pri kojem se rekurzija prekida i prelazi se na standardni iterativni algoritam.

```

func strassen(A, B, C Matrix) {
    n := len(A)

    if n <= THRESHOLD {
        multiplyMatrix(A, B, C)
    } else {
        matrices := computeMatricesParallel(A, B, n)
        combineResult(matrices, C)
    }
}

```

Listing 17: Go - funkcija *strassen*

Za matrice većih dimenzija, izračunavanje sedam pomoćnih matrica $M_1, \dots, M_7$, karakterističnih za Štrassenov algoritam, delegira se funkciji *computeMatricesParallel* [Listing 18]. Ova funkcija najpre podeli ulazne matrice na četiri podmatrice korišćenjem funkcije *splitView*, a zatim izračuna $S_1, \dots, S_{10}$, koje predstavljaju sume i razlike podmatrica ulaznih matrica. Pomoću tih matrica, rekursivnim pozivanjem Štrassenovog algoritma, izračunavaju se sedam pomoćnih matrica.

Svaki rekurzivni poziv realizovan je u posebnoj gorutini, dok je sinhronizacija obezbeđena pomoću *sync.WaitGroup*.

```

func computeMatricesParallel(A, B Matrix, n int) [7]Matrix {
    a11, a12, a21, a22 := splitView(A)
    b11, b12, b21, b22 := splitView(B)

    // Inicijalizacija S matrica
    ...
    // Racunanje S matrica
    sub(b12, b22, S[0])
    ...
    add(b11, b12, S[9])

    // Inicijalizacija M matrica
    ...

    var wg sync.WaitGroup
    wg.Add(7)
    go func() { strassen(a11, s[0], M[0]); wg.Done() }()
    ...
    go func() { strassen(s[8], s[9], M[6]); wg.Done() }()
    wg.Wait()

    return M
}

```

Listing 18: Go - funkcija *computeMatricesParallel*

Nakon što su sve pomoćne matrice izračunate, funkcija *combineResult* ih kombinuje u četiri kvadranta rezultujuće matrice $C_{11}, C_{12}, C_{21}, C_{22}$, u skladu sa formulama Štrasenovog algoritma [Listing 19].

```

func combineResult(m [7]Matrix, C Matrix) {
    c11, c12, c21, c22 := splitView(C)

    add(m[4], m[3], c11)
    sub(c11, m[1], c11)
    add(c11, m[5], c11)

    add(m[0], m[1], c12)

    add(m[2], m[3], c21)

    add(m[4], m[0], c22)
    sub(c22, m[2], c22)
    sub(c22, m[6], c22)
}

```

Listing 19: Go - Funkcija *combineResult*

4 Poređenje implementacija

U ovom poglavlju izvršeno je poređenje implementacija algoritama za množenje matrica u programskim jezicima Rust i Go. Poređenje obuhvata aspekte brzine i lakoće razvoja, performansi izvršavanja, kao i utrošak sistemskih resursa.

4.1 Brzina i lakoća razvoja

Sa aspekta razvoja softvera, Go se pokazao kao jezik sa jednostavnijom sintaksom i kraćim vremenom potrebnim za implementaciju algoritama. Podrška za višedimenzionalne nizove i ugrađeni mehanizmi za konkurentnost omogućavaju brzu realizaciju iterativnog i paralelnih algoritama.

Sa druge strane, Rust zahteva više početnog napora, posebno u delu upravljanja memorijom i životnim vekom podataka. Samostalna implementacija struktura *Matrix* i *MatrixView* uvela je veću kompleksnost koda, a time i povećala vreme potrebno za implementaciju algoritama. Prednosti ovakvog pristupa su se izrazili kroz omogućenu preciznu kontrolu nad rasporedom podataka u memoriji i izbegavanje nepotrebnog kopiranja.

Može se zaključiti da Go nudi veću produktivnost i kraće vreme razvoja, dok Rust zahteva više pažnje prilikom implementacije, ali zauzvrat pruža veću kontrolu nad niskonivojskim detaljima.

4.2 Vreme izvršavanja

Vreme izvršavanja predstavlja jedan od ključnih kriterijuma za poređenje implementacija algoritama uopšteno. U ovom radu analizirano je vreme izvršavanja iterativnog, zavadi-pa-vlada i Štrasenovog algoritma u oba programska jezika.

Merenja su izvršena za kvadratne matrice dimenzija $n \times n$, gde je n stepen broja 2. Vreme izvršavanja izraženo je u milisekundama i predstavlja prosečno vreme dobijeno tokom testiranja. Rezultati su prikazani tabelarno i grafički.

4.2.1 Iterativni algoritam

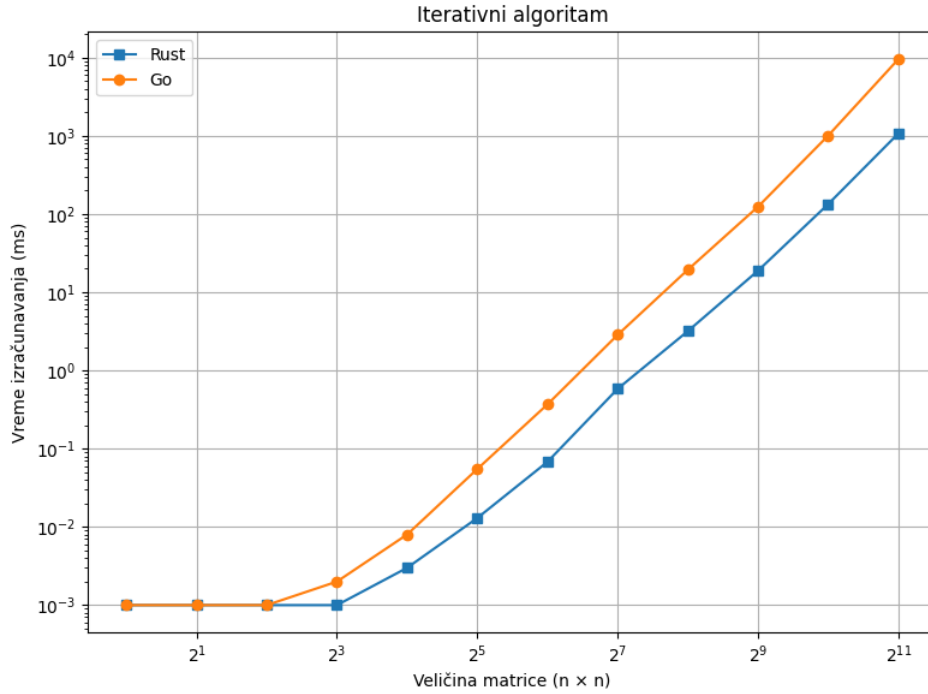
Tabela 1 prikazuje vreme izvršavanja standardnog iterativnog algoritma za množenje matrica u jezicima Rust i Go.

Jezik	$n = 8$	$n = 16$	$n = 32$	$n = 64$	$n = 128$	$n = 256$	$n = 512$	$n = 1024$	$n = 2048$
Rust	0.001	0.003	0.013	0.068	0.583	3.188	18.886	134.175	1074.674
Go	0.002	0.008	0.055	0.369	2.858	19.427	124.293	1007.876	9567.425

Tabela 1: Vreme izvršavanja iterativnog algoritma (ms)

Na osnovu tabele se može primetiti da, kako dimenzija matrice raste, razlike performansi u implementacijama postaju izraženije.

Grafički prikaz rezultata dat je na slici 1.



Slika 1: Grafički prikaz vremena izvršavanja iterativnog algoritma

4.2.2 Zavadi-pa-vladaj algoritam

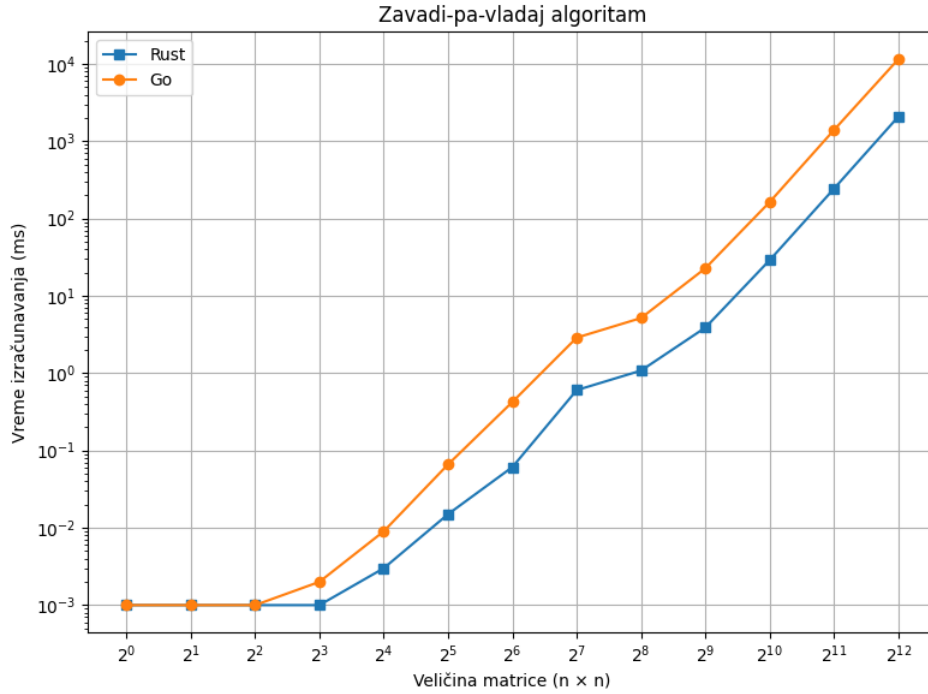
Tabela 2 prikazuje rezultate merenja za zavadi-pa-vladaj algoritam.

Jezik	$n = 16$	$n = 32$	$n = 64$	$n = 128$	$n = 256$	$n = 512$	$n = 1024$	$n = 2048$	$n = 4096$
Rust	0.003	0.015	0.061	0.603	1.079	3.884	28.94	243.258	2076.236
Go	0.009	0.066	0.424	2.877	5.159	22.719	163.247	1395.384	11478.027

Tabela 2: Vreme izvršavanja zavadi-pa-vladaj algoritma (ms)

Za manje dimenzije matrica performanse su uporedive sa iterativnim algoritmom, budući da se zbog praga THRESHOLD koristi iterativna verzija. Međutim, kako dimenzija matrice raste, paralelizacija počinje da ima značajan uticaj na ukupno vreme izvršavanja.

Grafički prikaz rezultata dat je na slici 2.



Slika 2: Grafički prikaz vremena izvršavanja zavadi-pa-vladaj algoritma

Rust implementacija pokazuje bolju skalabilnost i manji rast vremena izvršavanja u poređenju sa Go-om. Razlika postaje posebno izražena kod najvećih testiranih dimenzija, gde Go implementacija beleži značajno veće vreme izvršavanja.

4.2.3 Štrassenov algoritam

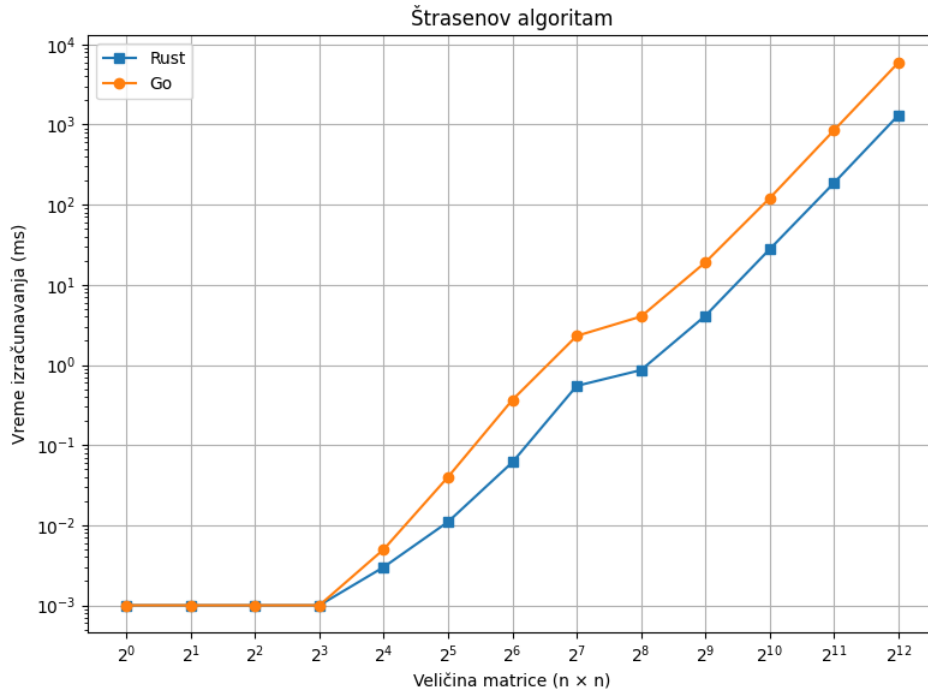
Rezultati merenja za Štrassenov algoritam prikazani su u tabeli 3.

Jezik	$n = 16$	$n = 32$	$n = 64$	$n = 128$	$n = 256$	$n = 512$	$n = 1024$	$n = 2048$	$n = 4096$
Rust	0.003	0.011	0.061	0.548	0.863	4.11	27.84	187.05	1307.511
Go	0.005	0.04	0.366	2.303	4.022	19.083	120.638	849.75	5931.847

Tabela 3: Vreme izvršavanja Štrassenovog algoritma (ms)

Štrassenov algoritam pokazuje najbolje performanse za velike dimenzije matrica u poređenju sa prethodna dva algoritma, što je očekivano s obzirom na smanjenu asimptotsku složenost.

Grafički prikaz rezultata dat je na slici 3.



Slika 3: Grafički prikaz vremena izvršavanja Štrasenovog algoritma

4.2.4 Diskusija rezultata

Na osnovu prikazanih rezultata može se zaključiti da za male dimenzije matrica razlike u performansama između jezika Rust i Go nisu značajne. Kako dimenzije matrica rastu, razlika između performansi postaje sve značajnija. Rust implementacije postižu manje vreme izvršavanja u sva tri algoritma.

Razlike u vremenu izvršavanja između implementacija mogu se objasniti razlikama u kompajlerima i načinu izvršavanja. Rust koristi LLVM kompajlerski okvir, koji omogućava agresivne optimizacije, uključujući uklanjanje provera granica i efikasnije korišćenje procesorskog keša u optimizovanim verzijama programa [4]. Sa druge strane, Go koristi sopstveni kompajler i izvršava se unutar Go *runtime*-a, što uvodi dodatne troškove vremena izvršavanja, kao što je provera granica indeksiranja nizova [5, 6].

Kod paralelnih algoritama, razlike nastaju zbog načina upravljanja paralelizmom. Rust implementacija koristi biblioteku *Rayon*, koja koristi fiksni skup niti, čime se ograničava broj istovremenih izvršavanja i smanjuje trošak kreiranja i sinhronizacije zadataka [7]. Nasuprot tome, Go implementacija koristi veliki broj gorutina koje se dinamički raspoređuju, što pri dubokoj rekurziji dovodi do povećanog vremena izvršavanja i slabije skalabilnosti za velike dimenzije matrica [5].

4.3 Utrošak resursa

Pored vremena izvršavanja i lakoće razvoja, važno je analizirati i utrošak sistemskih resursa tokom izvršavanja algoritama. U ovom poglavlju razmatra se potrošnja procesorskog vremena i memorije za implementacije u programskim jezicima Rust i Go.

Merenja su izvršena korišćenjem *Linux* alata `/usr/bin/time`, koji omogućava detaljan uvid u ponašanje programa tokom izvršavanja. Za potrebe ovog rada

analizirani su utrošak zauzeća procesora, kao i maksimalna zauzeta radna memorija.

4.3.1 Utrošak zauzeća procesora

Utrošak zauzeća procesora je analiziran procentualno i daje uvid u stepen paralelizacije algoritma.

Tabela 4 prikazuje procentualno zauzeće procesora za svaku implementaciju algoritama u oba programska jezika.

Jezik	Iterativni	Zavadi-pa-vladaj	Štrasenov
Rust	99	1304	1146
Go	100	1514	1379

Tabela 4: Utrošak zauzeća procesora (%)

Rezultati pokazuju da paralelni algoritmi ostvaruju značajno veći procenat zauzeća procesora u odnosu na iterativni algoritam, što potvrđuje efikasno korišćenje više procesorskih jezgara.

Iako Go implementacije pokazuju veći procenat zauzeća procesora, ukupno vreme izvršavanja paralelnih algoritama u Rust-u je kraće, što ukazuje da Rust efikasnije iskorišćava dostupne procesorske resurse.

4.3.2 Utrošak memorije

Utrošak memorije analiziran je na osnovu maksimalne zauzete radne memorije tokom izvršavanja programa. Ova metrika daje realnu sliku memorijskih zahteva implementiranih algoritama.

Tabela 5 prikazuje maksimalno zauzeće radne memorije za svaku implementaciju algoritama u oba programska jezika.

Jezik	Iterativni	Zavadi-pa-vladaj	Štrasenov
Rust	56.51	764.72	1823.41
Go	207.2	8064.42	13548.54

Tabela 5: Maksimalno zauzeće radne memorije (MB)

Rezultati pokazuju da paralelni algoritmi zahtevaju više memorije u poređenju sa iterativnim algoritmom, što je očekivano zbog kreiranja dodatnih podmatrica i rekrzivnih struktura. Štrasenov algoritam ima najveći memorijski utrošak zbog potrebe za čuvanjem pomoćnih matrica.

Rust implementacije koriste značajno manje memorije u odnosu na Go implementacije za isti algoritam, što se može pripisati boljem upravljanju memorijom i odsustvom *garbage collector*-a.

5 Zaključak

U ovom radu analizirana je implementacija i poređenje tri algoritma za množenje matrica: standardnog iterativnog algoritma, zavadi-pa-vladaj algoritma i Štrasenovog algoritma, u programskim jezicima Rust i Go.

Rezultati pokazuju da Rust implementacije u ostvaruju kraće vreme izvršavanja i manji memorijski utrošak u odnosu na Go, dok paralelni algoritmi značajno ubrzavaju izvršavanje u odnosu na iterativni pristup. Zavadi-pa-vladaj i Štrasenov algoritam koriste više memorije zbog kreiranja dodatnih podmatrica i pomoćnih struktura, ali omogućavaju efikasno korišćenje više procesorskih jezgara.

Sa aspekta razvoja, Go pruža jednostavniji model paralelizacije i upravljanja konkurentnim izvršavanjem, dok Rust zahteva eksplicitno upravljanje memorijom i sistemskim nitima, ali omogućava bolju iskorišćenost resursa. Zaključno, izbor algoritma i programskog jezika predstavlja kompromis između performansi i jednostavnosti implementacije.

Literatura

- [1] Matrix Operations: www.cuemath.com
- [2] Divide and Conquer: www.staff.univ-batna2.dz
- [3] T. Nguyen, A. Adamson, A. Santucci (2016): Distributed Algorithms and Optimization
- [4] The Rust Programming Language: www.rust-lang.org
- [5] A Guide to the Go Garbage Collector: www.go.dev
- [6] BCE (Bound Check Elimination): www.go101.org
- [7] J. Mo: Data Parallelism with Rust and Rayon: www.shuttle.dev